

PRODUCT & ARCHITECTURE DOCUMENTATION

NEXORA

Next-generation enterprise operations

Enterprise ERP Platform — complete technical and product reference

Covers identity, architecture, security, modules, phased delivery, and governance

Version 1.0 | Master Documentation

Formerly developed under the working name SutraERP · September 2026

1. About This Document

This is the master reference for NEXORA — what it is, why it is built the way it is, and how it will be delivered. It documents the product identity and design system, the full technical architecture, security and compliance posture, the complete module surface, and the phase-by-phase development plan, in one place.

This is documentation, not a pitch: every section states what has been decided, what is deferred and why, and what remains open. Our organization is both the author of this platform and its first production user — Phase 1 of the roadmap in Section 14 replaces our own dependency on our existing Zoho ERP environment, while the architecture underneath is built to extend into a commercial multi-tenant product.

2. Product Identity

2.1 Name & Meaning

Element	Definition
Product name	NEXORA
Meaning	NEXORA combines “nexus” — a hub where connections converge — with a forward-looking suffix evoking a new horizon. It names what the platform actually is: a converging hub for every enterprise process, built for the next generation of enterprise operations.
Tagline	Next-generation enterprise operations
Category	Configurable Process-to-Ledger ERP Platform
Market position	Between heavyweight enterprise suites (SAP, Oracle, Dynamics 365, Workday) and lightweight SMB tools (Zoho, Odoo) — for organizations that have outgrown basic business tools but do not need SAP-level cost and complexity.

2.2 Design System — Color Palette

The palette below is applied consistently across the product interface and every piece of NEXORA documentation, including this one: headings in Nexora Navy, accents in Aurora Gold, approvals and success states in Signal Teal, and alerts in Alert Coral.

NEXORA — Color System

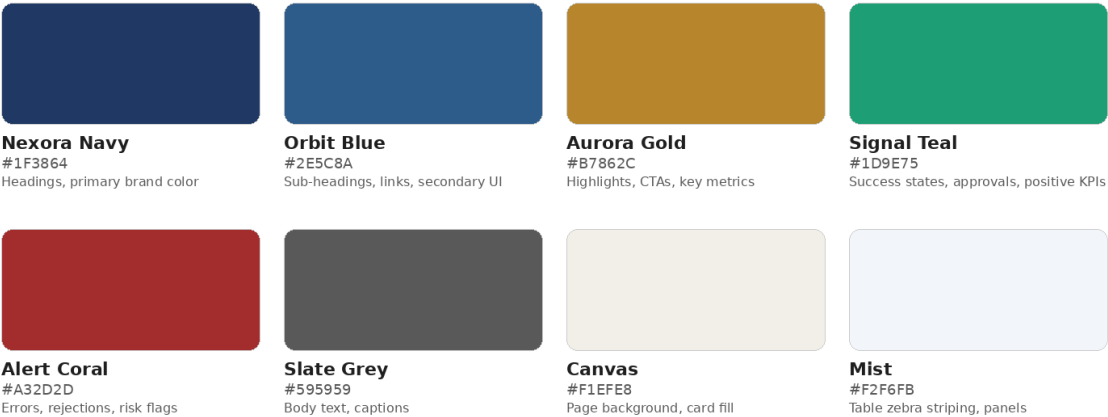


Figure 1 — NEXORA color system with usage notes

2.3 Interface Reference

An illustrative rendering of the employee dashboard, showing the palette applied to real interface elements: KPI cards, an approval queue, and a ledger snapshot.

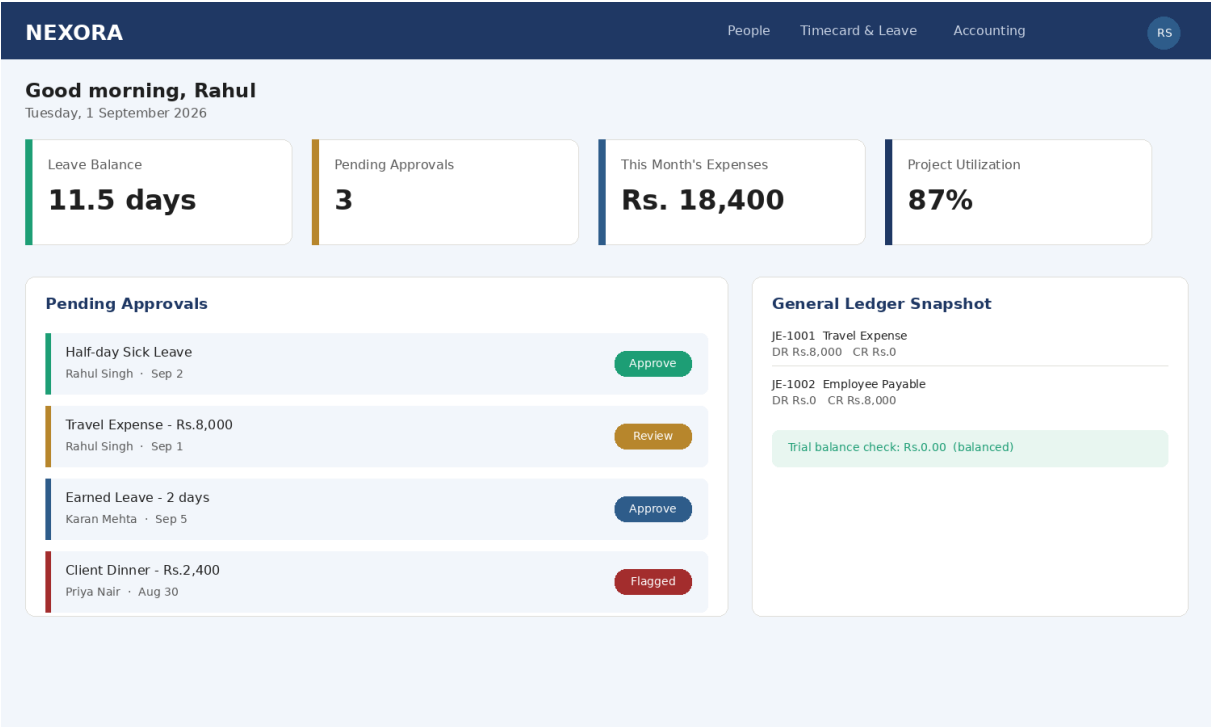


Figure 2 — Illustrative dashboard view in the NEXORA visual language

3. Product Vision & Philosophy

Vision: a modern, modular, configurable ERP platform that connects people, processes, finance, projects, and customers through one secure source of truth — where an organization activates only the modules it needs, and nothing more.

- One source of truth — no duplicate departmental databases.
- Integration first — every module can communicate with every other module by design, not by later patchwork.
- Configuration over customization — workflows, approvals, roles, policies, and tax rules are configured by an administrator, not coded by an engineer.
- API-first — every capability is reachable through an API, not locked behind a screen.
- Security by design — built into the architecture from the first commit, not added before a compliance audit.
- Mobile and web from day one — employees, managers, finance, and administrators, on whichever device they are using.
- Product first — every internal requirement is tested against one question: is this organization-specific, or should it become a configurable platform capability?

4. Unique Value Proposition

Claims like “affordable ERP”, “AI-powered ERP”, or “modular ERP” are used by nearly every competitor and defend nothing. NEXORA's differentiation is architectural.

Differentiator	What It Means
Configurable process-to-ledger engine	Any business action — expense, leave, purchase, invoice — moves through the same policy → validation → approval → transaction → ledger → audit pipeline. A new request type is added without rebuilding the engine.
Configuration-first, not code-first	An administrator changes an approval rule (e.g. “expenses over Rs.50,000 require Director sign-off”) through a workflow builder — no developer, no deployment.
Internal-first validation	Real employees, real transactions, real accounting, before any external customer sees the product — producing a proven reference implementation rather than a theoretical one.
Process intelligence	The platform does not only record “approved” — over time it can identify why an approval took four days and where the bottleneck sits.
Open integration architecture	REST APIs, webhooks, events, and an SDK from the first release — no vendor lock-in for organizations that adopt it.

Explicitly Not Claimed

- Not positioned as “cheapest ERP”, “most powerful ERP”, “SAP replacement”, or “AI-first ERP” — these are weak, overused, or indefensible claims that invite an easy rebuttal.

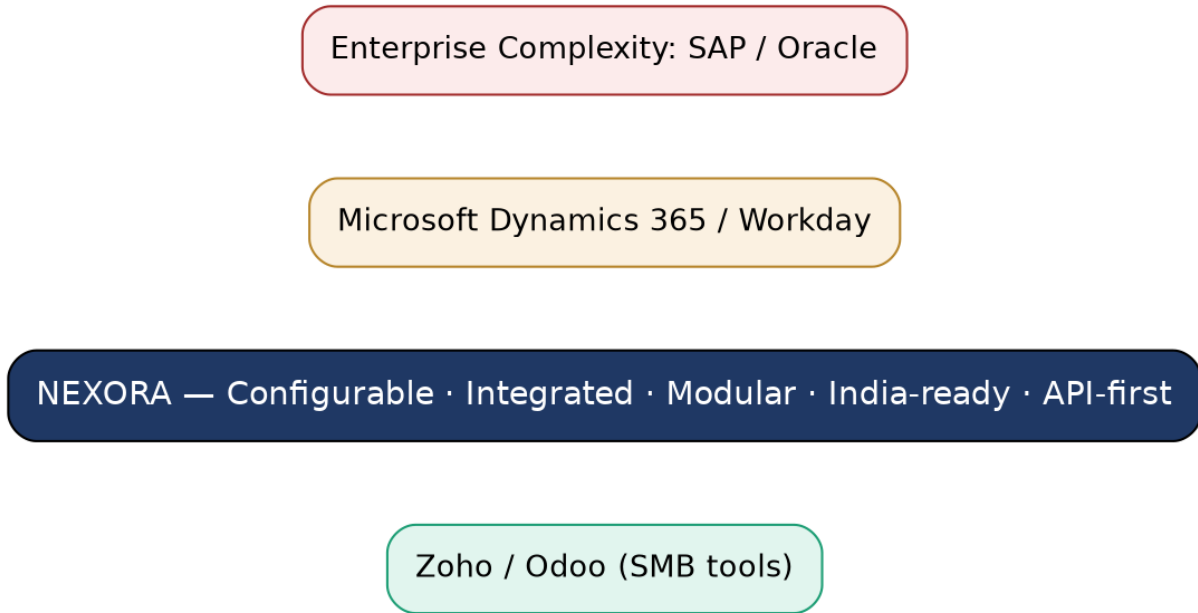


Figure 3 — Competitive positioning: deliberately between enterprise complexity and SMB tooling

5. Competitive Capability Reference

A candid comparison of Phase 1 capability against the current ERP landscape. This is a reference for internal decision-making, not a marketing claim — NEXORA is not expected to match enterprise suites feature-for-feature at this stage.

Capability	NEXORA (current)	Enterprise Suites (SAP/Oracle/Dynamics)	Workday	Zoho / Odoo
Employee master	Yes	Yes	Yes	Yes
Leave & attendance	Yes	Yes	Yes	Yes
Expenses	Yes	Yes	Yes	Yes
Accounting / GL	Partial (Phase 3)	Yes	Yes	Yes
Project costing	Yes	Yes	Partial	Yes
Full project management	Planned (Phase 4)	Yes	Partial	Yes
Procurement	Planned (Phase 5)	Yes	Partial	Yes
Inventory	Planned (Phase 6)	Yes	Limited	Yes
Payroll	Planned (Phase 11)	Strong	Strong	Partial
Sales / CRM	Planned (Phase 7)	Yes	Partial	Yes
Multi-tenant SaaS	Architected, not yet live	Yes	Yes	Yes
Approval engine	Strong	Strong	Strong	Strong
Immutable ledger design	Strong	Strong	Strong	Partial

6. Technology Stack

Layer	Technology	Rationale
Web frontend	React + TypeScript	Enterprise ERP workflows require dense data grids, bulk operations, and keyboard-driven navigation — better served by React than a single cross-platform toolkit for desktop/web.
Mobile frontend	Flutter + Dart	One codebase for iOS and Android, suited to the simplified, touch-first workflows mobile users need: approvals, leave, expense capture, notifications.
Backend	.NET (current LTS) — ASP.NET Core, Entity Framework Core, C#	Strong typing, a mature security ecosystem, and a large enterprise hiring pool. The current supported LTS is targeted at build time rather than a fixed version frozen in advance.
Architecture style	Modular monolith, initially	Clear module boundaries (Finance, HR, Projects) inside one deployable unit; independent services are extracted later only where scale justifies the added operational complexity.
Database	PostgreSQL	Relational integrity, transactions, JSON support, and no per-seat licensing — the schema is designed around Tenant → Company → Business Unit → Department from the first migration.
Cache	Redis	Sessions, distributed caching, rate limiting.
Messaging	Background workers initially; RabbitMQ or Azure Service Bus once volume justifies it	A message broker is introduced when integration patterns actually require it, not by default.
File storage	Object storage (Azure Blob / S3-compatible)	The database stores metadata only; large files never live directly inside PostgreSQL.
Identity & access	OAuth 2.0 + OpenID Connect, MFA, SSO, RBAC + policy-based (ABAC) authorization	Role alone is insufficient for authorization; department, reporting line, project, and company scope are all considered.
DevOps	Git, CI/CD, Docker, infrastructure as code, automated testing, staging → UAT → production	Every release passes a security scan before it reaches production.
Observability	Structured logs, metrics, distributed tracing, alerting, audit logging	Distinct from the audit trail — this is how the operations team learns the system is degrading before employees notice.
Analytics	PostgreSQL reporting layer initially, evolving to a data warehouse/lakehouse at scale	Dashboards never query transactional tables directly.

7. System Architecture

Every request — from a React or Flutter client down to the database — passes through the security boundary and the platform's core services before reaching any business module.

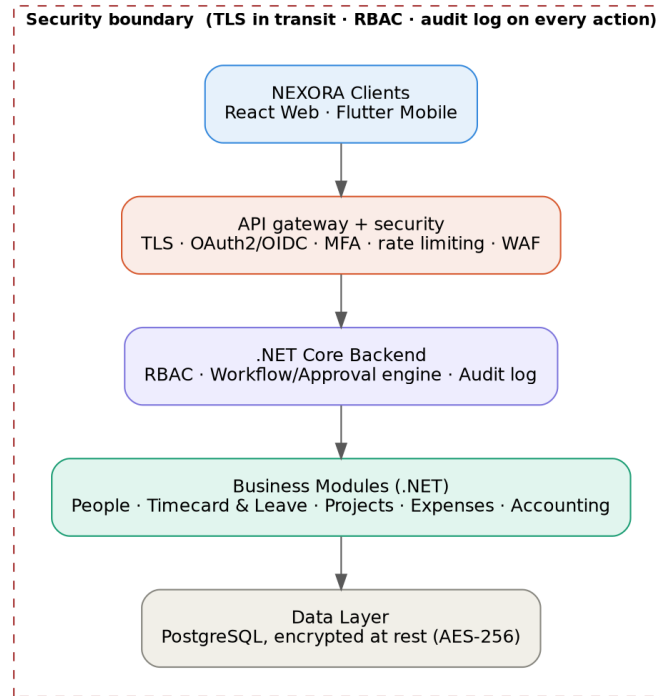


Figure 4 — End-to-end request path through the security boundary

7.1 Platform Layers

The platform organizes into three layers on top of a common core: Platform Services (identity, workflow, audit), Business Modules (Finance, HR, Projects/SCM), and an Intelligence Layer (BI, KPIs, AI) that reads from all of them.

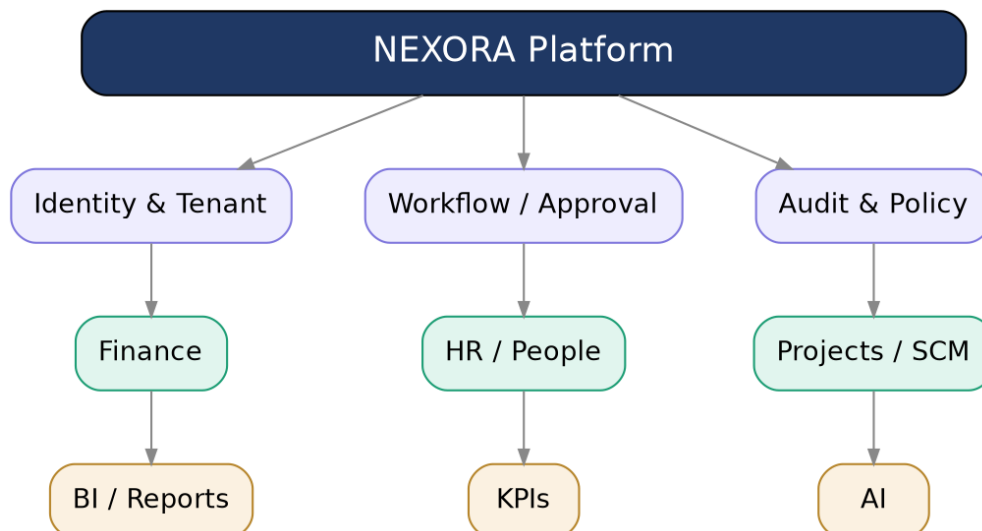
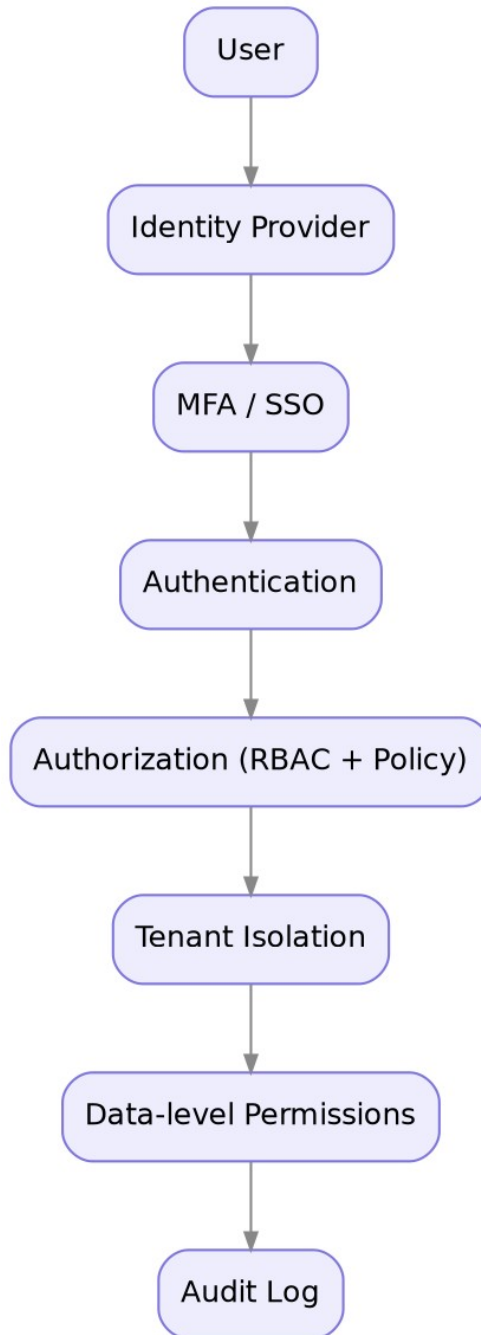


Figure 5 — NEXORA platform layers

7.2 Security Chain

Every request is authenticated and authorized through the same chain regardless of which module it touches. Role alone never determines access — tenant, department, and data scope apply on every request.

*Figure 6 — Identity through to audit, applied uniformly*

7.3 Organizational Data Model

Every table in the schema is designed against this hierarchy from the first migration, even while only one organization uses the platform. Retrofitting tenant isolation after the fact is one of the most expensive mistakes an ERP team can make — so the hierarchy exists in the data model well before multi-tenant rollout is commercially needed.



Figure 7 — Organizational hierarchy embedded in every record

7.4 Multi-Tenancy Approach

Multi-tenancy implementation — tenant sign-up, billing, plan tiers — can wait until external customers are ready to onboard. Multi-tenancy architecture cannot wait; it is designed into the schema from Phase 1.

- Tenant-specific configuration, roles, workflows, branding, localization, and module activation are all modeled from the start.
- A single platform codebase serves every tenant; isolation is enforced at the data layer, not by separate deployments per customer.

8. Platform Core Services

Six reusable engines sit beneath every business module. None of them is specific to Leave or Expense or any single module — building them once here is what prevents the same logic being re-implemented five times across the platform.

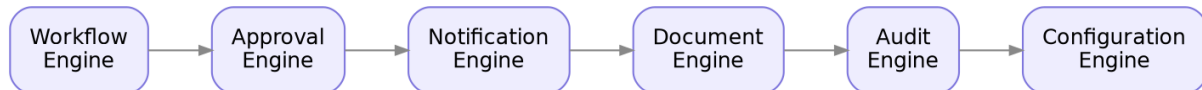


Figure 8 — The six core services shared by every business module

Engine	Responsibility
Workflow Engine	Defines the sequence a business object moves through (submitted → validated → approved → posted), reusable across Leave, Expense, Purchase, Invoice, and future object types.
Approval Engine	Resolves who must approve a given request based on role, amount, department, or project — configurable per rule, not hard-coded per module.
Notification Engine	Email, in-app, and push notifications today; SMS integration planned. Handles reminders and escalations when an approval sits idle.
Document Engine	Attaches and stores receipts, contracts, employee documents, and compliance documents against any record in the platform.
Audit Engine	Records who did what, when, where, and what changed, for every sensitive operation — logins, permission changes, master-data edits, approvals, financial transactions, configuration changes.
Configuration Engine	Lets an administrator define departments, leave types, approval rules, expense policies, roles, and custom fields without a code change or deployment.

9. Security & Compliance

Category	Controls
Authentication	MFA, SSO, OAuth2/OIDC, session management, refresh-token security, account lockout policies.
Authorization	RBAC combined with policy-based (ABAC) rules; resource-level and department-level data access; approval-authority limits per role.
Audit	Login and logout, failed authentication attempts, permission changes, master-data changes, approval changes, financial transactions, and configuration changes are all logged.
Infrastructure	Secrets management, key management, intrusion detection, vulnerability scanning, dependency scanning.
Data protection	Encryption in transit (TLS) and at rest, PII protection, data retention policy, data export and deletion rules.
Backup & disaster recovery	Defined Recovery Point Objective and Recovery Time Objective targets, encrypted backups, scheduled restore testing, and a documented failover strategy — not simply “daily backup.”
Compliance	India-first statutory support (Provident Fund, ESIC, Professional Tax, TDS) is designed as a configurable localization layer rather than logic hard-coded into the core.

10. Integration Architecture

An ERP rarely operates in isolation. The integration layer exists from Phase 1, even though most external integrations themselves are built later.

- REST APIs, versioned from the first release, so breaking changes never silently affect an existing integration.
- Webhooks and asynchronous events for systems that need to react to NEXORA activity without polling.
- Idempotency and retry handling built into the integration layer, so a duplicated request never creates a duplicated transaction.
- API keys and OAuth-based access for external systems, with full integration logging.
- Planned downstream integrations: payroll providers, banking, email, SSO providers, CRM, tax systems, payment gateways, document storage, and BI tools.

11. Reporting & Analytics Architecture

Dashboards are never allowed to run heavy queries directly against transactional tables — a dedicated reporting layer exists specifically to keep operational performance and reporting performance independent of each other.

- HR reporting: headcount, attrition, leave utilization, attendance, overtime.
- Finance reporting: cash flow, expense trends, budget variance, aging, reimbursement status.
- Project reporting: budget vs. actual, cost by employee, billable utilization, project profitability.
- Management reporting: a company-wide dashboard combining financial and operational KPIs across departments.

12. Full Module Reference

The complete functional surface NEXORA is designed to eventually cover, grouped by domain. Phase 1 (Section 14) implements only a slice of this deliberately — the remainder is architected for, not built prematurely.

Module Group	Covers
A — Platform Core	Identity & access, organization/tenant management, workflow & approval engine, notifications, documents, audit & compliance.
B — People / HCM	Employee master, attendance, leave, timesheets, expenses (Phase 1); recruitment, payroll, performance, benefits (future).
C — Finance	General ledger, chart of accounts, fiscal periods, AP/AR, banking, budgeting, fixed assets, tax.
D — Expense Management	Claims, receipts, policy validation, approvals, reimbursement, accounting integration.
E — Project Management	Project/task/milestone planning plus project financials — time, expense, resource cost, revenue, profitability.
F — Procurement	Requisition → approval → RFQ → purchase order → goods receipt → invoice → 3-way match → payment.
G — Inventory & Warehouse	Products, warehouses, stock movement, transfers, batches, serials, reorder levels.
H — Sales & CRM	Leads, opportunities, quotations, sales orders, delivery, invoicing, payment.
I — Supply Chain	Demand planning, supplier management, logistics, forecasting, replenishment.
J — Manufacturing	BOM, work orders, MRP, quality, maintenance — optional, gated behind market validation.
K — Asset & Facility Management	Asset register, assignment, maintenance, depreciation, disposal.
L — Analytics & BI	Operational and management dashboards, forecasting, anomaly detection.
M — AI	Document/receipt extraction, expense classification, forecasting, and eventually agentic workflows — applied to solve real ERP problems, not added for its own sake.

13. Phase-Wise Development Plan

Fifteen phases, grouped into four delivery stages. Phases 0-4 build the platform core and validate it internally; Phases 5-9 expand into broader business operations; Phases 10-12 harden the platform for enterprise use; Phases 13-14 turn it into a commercial SaaS product.

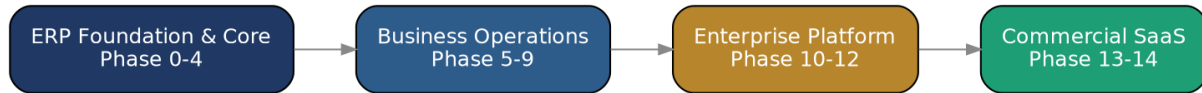


Figure 9 — The fifteen phases grouped into four delivery stages

Phase	Goal	Key Scope	Deliverable / Gate
0 — Discovery & Blueprint	Understand our needs and the future product's needs together	Zoho capability inventory, process mapping, competitor benchmarking, security & compliance requirements	Architecture and product strategy approved
1 — ERP Foundation	Build the platform every future module depends on	Identity, tenant/org model, workflow engine, approval engine, notification engine, audit engine, API foundation	Working platform shell
2 — People + Time + Leave	Replace the first major Zoho workflows	Employee master, departments, reporting hierarchy, attendance, timesheets, leave (including half-day and sick leave rules), self-service	Employees log time and apply leave in NEXORA
3 — Expense + Finance Core	Money starts flowing through the ledger	Expense claims/approval/reimbursement; GL, chart of accounts, journals, AP/AR, budgets, fiscal periods, immutable posting with reversal	An approved expense posts a balanced, immutable journal entry
4 — Project Management + Financials	Connect time and expense to project profitability	Projects, tasks, milestones, resources, budgets; time + expense + resource cost + revenue = profitability	Live budget-vs-actual per project, no spreadsheets
5 — Procurement	Close the largest Phase-1 gap: buying from suppliers	Requisition → approval → RFQ → PO → goods receipt → invoice → 3-way match → payment	A purchase can be requested, approved, and paid end-to-end
6 — Inventory & Warehouse	Track stock, not just cost	Product master, stock, warehouses, transfers, adjustments, batch/serial, reorder levels	Stock levels reconcile against transactions
7 — Sales & CRM	Bring revenue into the same engine	Lead → opportunity → quotation → order → delivery → invoice → payment, integrated with inventory and projects	A sale flows to accounting with zero manual entry
8 — Supply Chain	Plan ahead, not just react	Demand planning, supplier management, logistics, forecasting, replenishment	Supply plan reflects real demand signals
9 — Asset & Facility Management	Track what the company owns	Asset register, assignment, maintenance, depreciation, disposal	Every fixed asset has a full lifecycle record
10 — Advanced Finance	Enterprise-grade financial control	Advanced budgeting, forecasting, cash flow, multi-company, multi-currency, intercompany, financial close	Multi-entity close completes on schedule
11 — Advanced HR	Full HCM, not only leave and attendance	Recruitment, onboarding, payroll (including PF/ESIC/PT/TDS), performance, training, workforce analytics	Payroll runs inside NEXORA, statutory-compliant
12 — Manufacturing (optional)	Only if commercial demand justifies it	BOM, MRP, work orders, production costing, quality, maintenance	Gated behind explicit market validation
13 — BI + AI	Move from reporting to explanation	Operational → management → predictive analytics; document intelligence, anomaly detection, forecasting, early agentic workflows	A dashboard can explain “why”, not just “what”
14 — Commercial SaaS Platform	Become externally sellable	Multi-tenancy, subscription plans (Starter/Professional/Enterprise), module licensing, self-serve tenant onboarding	A new customer reaches their first transaction with no engineering involvement

Phase 1 of the roadmap corresponds to the four modules already agreed for the immediate build — People, Timecard & Leave, Accounting/Reimbursement, and Project Expenses — delivered here as Phases 1-4 with the added rigor of an immutable ledger, fiscal periods, and full project financials.

14. Definition of Done — Worked Example

Every module in this roadmap is only considered complete when it satisfies measurable, testable acceptance criteria — not a one-line description. The Leave module is used here as a worked example of the standard applied to every module.

- Employee can submit a leave request, including half-day and backdated sick leave.
- Leave balance is calculated correctly, including partial-day debits.
- Overlap and blackout-date conflicts are detected before submission.
- Manager approval and rejection both function and are logged.
- Cancellation of an already-submitted request is supported.
- Concurrent requests against the same balance are handled without a race condition.
- Historical balances are preserved even after an organizational change (department, manager).
- Every state transition (submit, approve, reject, cancel) writes an audit entry.
- Notifications fire correctly on every state transition.

Functional, technical (unit/integration/API/performance tests), security, business (UAT passed, users trained, documentation available), and production (monitoring, backups, rollback plan) criteria are all required together — a module is not “done” on functional criteria alone.

15. Data Governance & Migration Strategy

Since our organization is the first production customer, migrating cleanly off the existing Zoho environment is itself a first-class deliverable, not an afterthought.

- Stage 1 — Inventory every existing Zoho process: data, users, roles, approvals, reports, integrations, and pain points, not just features.
- Stage 2 — Build AS-IS and TO-BE process maps for each workflow being migrated.
- Stage 3 — For each process, decide: replicate, improve, remove, automate, or generalize into a configurable capability.
- Stage 4 — Migrate: extract, transform, clean, validate, map, and import — never a raw CSV export and import.
- Stage 5 — Run NEXORA and Zoho in parallel to validate before cutover.
- Stage 6 — Cut over to NEXORA as the system of record.
- Stage 7 — Zoho becomes a read-only archive per an approved retention policy.

16. Success Metrics

Category	Metrics
Adoption	Greater than 90% of targeted users active; manual spreadsheet usage down; duplicate data entry down.
Process efficiency	Approval turnaround time reduced; reimbursement processing time reduced.
Financial	Accounting errors reduced; audit findings resolved without rework.
System health	API response time, error rate, uptime, and security incident count within target.
Product readiness	Percentage of functionality configurable without code; percentage of modules reusable across future tenants; time to onboard a new tenant.
Commercial (north star)	Time-to-First-Transaction — how quickly a new customer reaches their first real transaction after signup.

17. Risk Register

Risk	Mitigation
Scope explosion beyond Phase 1	Strict module gates — each phase has a defined exit gate before the next begins.
Zoho migration data issues	Dedicated migration plan with parallel validation before cutover (Section 15).
Accounting errors	Finance UAT plus the immutable-ledger-and-reversal design (Section 13, Phase 3).
Security failure	Full security architecture (Section 9) built from Phase 1, not retrofitted.
Hard-coded, customer-specific rules	Configuration engine — a requirement is only hand-coded if configuration genuinely cannot solve it.
Over-engineered microservices too early	Modular monolith first; independent services are extracted only where scale justifies it.
Product becomes internal-only	Explicit product governance (Section 18) and multi-tenancy architecture from day one.
Aggressive timeline	Phase gates use measurable acceptance criteria, not calendar promises.

18. Governance Model

This is a product, not a side project — it requires a steering body beyond the engineering team.

- ERP Steering Committee: Product Owner, Technical Architect, Finance Representative, HR Representative, Operations Representative, End-user Representatives.
- Flow: business requirement → product evaluation → architecture review → development → UAT → release.
- Every requirement is tested against one question before it is built: can configuration solve this, or does it need a genuine platform capability?

19. Frequently Challenged Questions

Questions this documentation is expected to face, answered directly rather than left implicit.

Question	Answer
Why build instead of buying Odoo, Dynamics, or SAP Business One?	We control the roadmap, data, and architecture, and can validate every workflow against our own operations before generalizing it into a sellable product — see Section 15.
Where is MFA and SSO?	Defined in Section 9 as part of the authentication layer, alongside OAuth2/OIDC.
Where is backup and disaster recovery?	Section 9 requires explicit RPO/RTO targets and tested restores, not a vague “daily backup” statement.
How does the ledger handle corrections?	Only through a linked reversal entry (Section 13, Phase 3) — postings are never edited or deleted.
How will multi-tenancy work?	Architected from Phase 1 into the data model (Section 7.3-7.4); implemented commercially in Phase 14.
What happens if Phase 1 fails to hit its timeline?	Each phase has a measurable deliverable/gate (Section 13); the timeline is indicative and subject to team capacity, not a fixed promise.
What is our unique advantage over existing ERP products?	The configurable process-to-ledger engine and configuration-first philosophy (Section 4) — not feature count.

20. Closing Statement

NEXORA is not an internal business application that might later become a product. It is a commercial ERP platform, with our organization serving as its first production environment.

Phase 1 solves our immediate operational requirements and ends our dependency on Zoho. Every subsequent phase expands the platform toward a complete ERP covering Finance, HR, Procurement, Inventory, Sales, Projects, Supply Chain, Assets, Analytics, and AI.

The platform's advantage is not the number of screens it has. It is the engine underneath those screens: one configurable pipeline connecting business rules, approvals, transactions, accounting, audit, and analytics into a single continuous thread — from the first policy check to the final ledger entry.